



赞同 3
分享

复杂推理模型从服务器移植到Web浏览器的理论和实战



阿里云开发者

已认证账号

关注

3 人赞同了该文章

简介：随着机器学习的应用面越来越广，能在浏览器中跑模型推理的Javascript框架引擎也越来越多了。在项目中，前端同学可能会找到一些跑在服务端的python算法模型，很想将其直接集成到自己的代码中，以Javascript语言在浏览器中运行。本文就基于pyodide框架，从理论和实战两个角度，帮助前端同学解决复杂模型的移植这一棘手问题。



作者 | 道仙⁺

来源 | 阿里技术公众号

一 背景

随着机器学习的应用面越来越广，能在浏览器中跑模型推理的Javascript框架引擎也越来越多了。在项目中，前端同学可能会找到一些跑在服务端的python算法模型，很想将其直接集成到自己的代码中，以Javascript语言在浏览器中运行。

对于一部分简单的模型，推理的前处理、后处理比较容易，不涉及复杂的科学计算，碰到这种模型，最多做个模型格式转化，然后用推理框架⁺直接跑就可以了。这种移植成本很低。

赞同 3



添加评论

分享

喜欢

收藏

申请转载

...

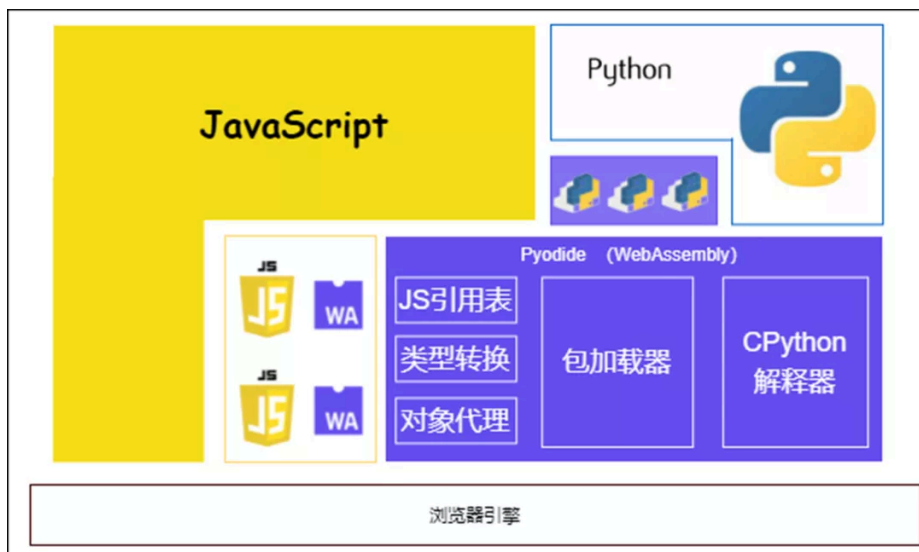


费力还容易出错。

Pyodide作为浏览器中的科学计算框架，很好的解决了这个问题：浏览器中运行原生的Python代码进行前、后处理，大量numpy⁺、scipy的矩阵、张量等计算无需翻译为JavaScript，为移植节省了很多工作。本文就基于pyodide框架，从理论和实战两个角度，帮助前端同学解决复杂模型的移植这一棘手问题。

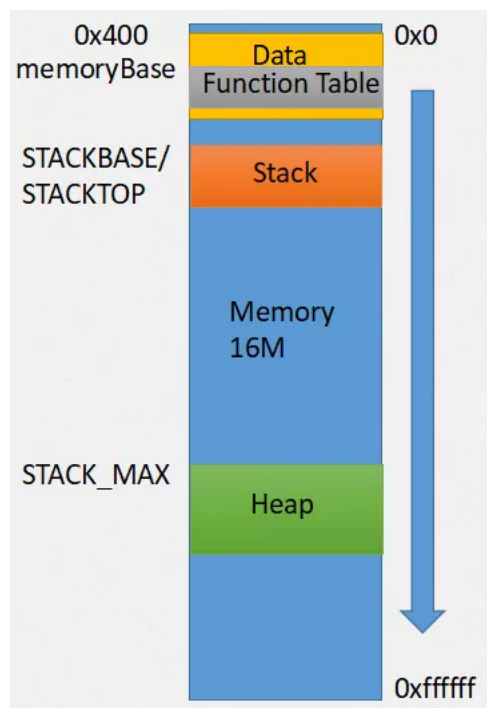
二 原理篇

Pyodide是个可以在浏览器中跑的WebAssembly（wasm）应用。它基于CPython的源代码⁺进行了扩展，使用emscripten编译成为wasm，同时也把一大堆科学计算相关的pypi包也编译成了wasm，这样就能在浏览器中解释执行python语句进行科学计算了。所以pyodide也必然遵循wasm的各种约束。Pyodide在浏览器中的位置如下图所示：



1 wasm内存布局

这是wasm线性内存的布局：

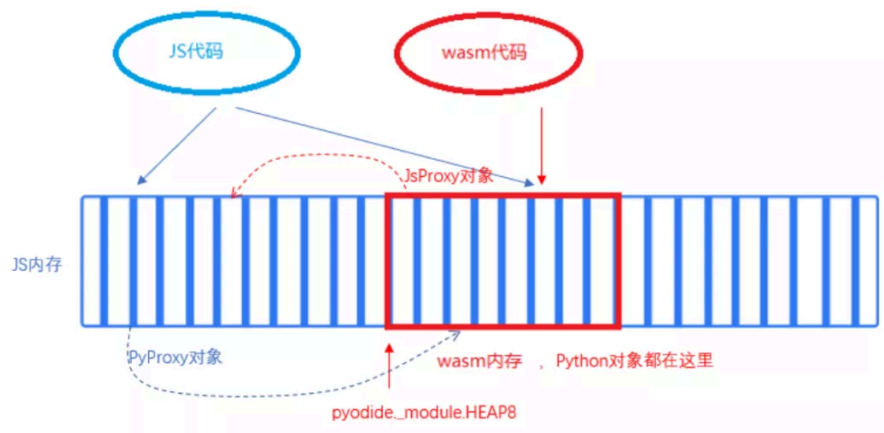


Data数据段是从0x400开始的，Function Table表也在其中，起始地址为memoryBase（Emscripten）。

2 Javascript与Python的互访

浏览器基于安全方面的考虑，防止wasm程序把浏览器搞崩溃，通过把wasm⁺运行在一个沙箱化的执行环境中，禁止了wasm程序访问Javascript内存，而Javascript代码却可以访问wasm内存。因为wasm内存本质上是一个巨大的ArrayBuffer，接受Javascript的管理。我们称之为“单向内存访问”。

作为一个wasm格式的普通程序，pyodide被调用起来后，当然只能直接访问wasm内存。



为了实现互访，pyodide引入了proxy，类似于指针：在Javascript侧，通过一个PyProxy对象来引用python内存里的对象；在Python侧，通过一个JsProxy对象来引用Javascript内存里的对象。

在Javascript侧生成一个PyProxy对象：

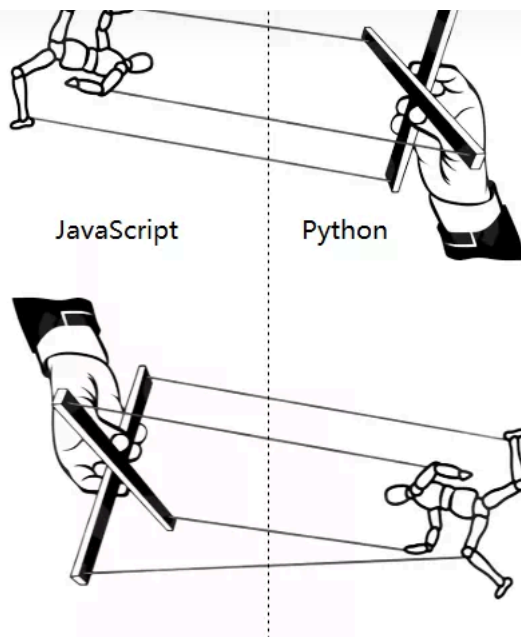
```
const arr_pyproxy = pyodide.globals.get('arr') // arr是python里的一个全局对象
```

在Python侧生成一个JsProxy对象：

```
import js
from js import foo # foo是Javascript里的一个全局对象
```

互访时的类型转换分为如下三个等级：

- **【自动转换】** 对于简单类型，如数字、字符串、布尔等，会被自动拷贝内存值，此时产生的就不是Proxy、而是最终的值了。
- **【半自动转换】** 非简单的内置类型，都需要通过to_js()、to_py()方式来显式转换：
 - 对于Python内置的list、dict、numpy.ndarray等对象，不属于简单类型，不会自动转换类型，必须通过pyodide.to_js()来转，相应的会被转成JS的list、map、TypedArray类型
 - 反过来也类似，通过to_py()方法，JS的TypedArray转为memoryview，list、map转为list、dict
- **【手动转换】** 各种class、function和用户自定义类型，因为对方的语言没有对应的现成类型，所以只能以proxy的形式存在，需要通过运算符来间接操纵，就像操纵提线木偶一样。为了达到方便操纵的目的，pyodide对两种语言进行了语法模拟，用一种语言里的操作符模拟另一种语言的类似行为。例如：JS中的let a=new XXX(), 在Python中就变为a=XXX.new()。



这里列举了一部分，详情可以查文档（见文章底部）。

Javascript	Python
<code>foo in proxy</code>	<code>hasattr(x, 'foo')</code>
<code>proxy.foo</code>	<code>x.foo</code>
<code>proxy.foo = bar</code>	<code>x.foo = bar</code>
<code>delete proxy.foo</code>	<code>del x.foo</code>
<code>Object.getOwnPropertyNames(proxy)</code>	<code>dir(x)</code>
<code>proxy(...)</code>	<code>x(...)</code>
<code>proxy.foo(...)</code>	<code>x.foo(...)</code>
<code>proxy.length</code>	<code>len(x)</code>

Javascript的模块也可以引入到Python中，这样Python就能直接调用该模块的接口和方法了。例如，pyodide没有编译opencv包，可以使用opencv.js：

```
import pyodide
import js.cv as cv2
print(dir(cv2))
```

这对于pyoc

我们从一个空白页面开始。使用浏览器打开测试页面（测试页面见文章底部）。

1 初始化python

为了方便观察运行过程，使用动态的方式加载所需js和执行python代码。打开浏览器控制台，依次运行以下语句：

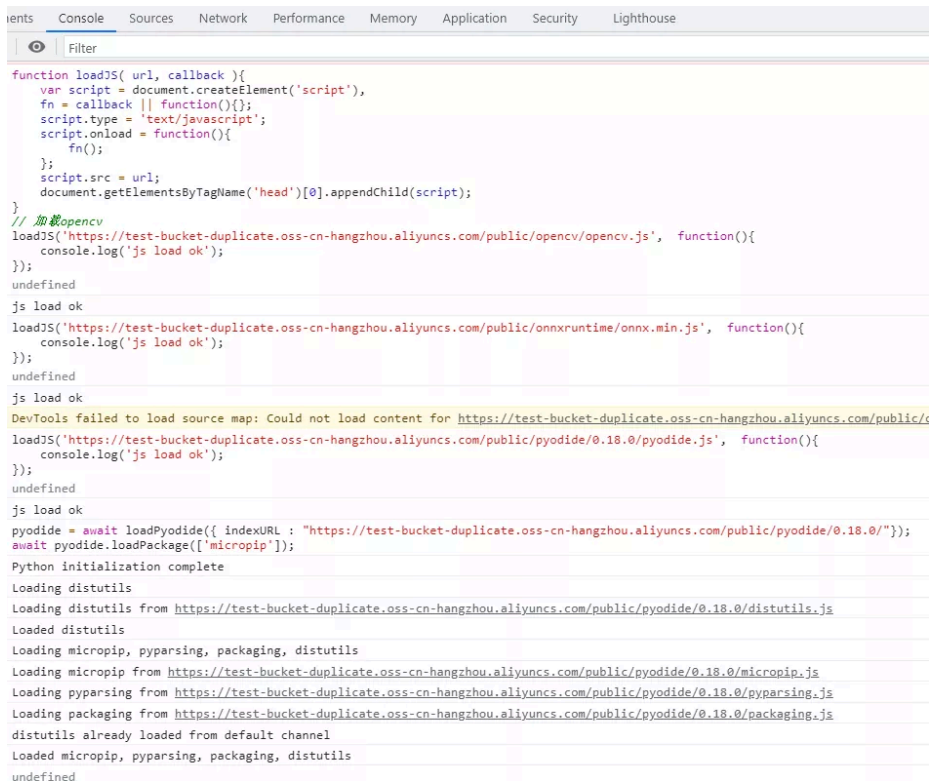
```
function loadJS( url, callback ){
    var script = document.createElement('script'),
    fn = callback || function(){};
    script.type = 'text/javascript';
    script.onload = function(){
        fn();
    };
    script.src = url;
    document.getElementsByTagName('head')[0].appendChild(script);
}

// 加载opencv
loadJS('https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/opencv/opencv.js', function(){
    console.log('js load ok');
});

// 加载推理引擎onnxruntime.js。当然也可以使用其他推理引擎
loadJS('https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/onnxruntime/onnxruntime.min.js', function(){
    console.log('js load ok');
});

// 初始化python运行环境
loadJS('https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/pyodide/pyodide.js', function(){
    console.log('js load ok');
});

pyodide = await loadPyodide({ indexURL : "https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/pyodide/0.18.0/" });
await pyodide.loadPackage(['micropip']);
```



```
function loadJS( url, callback ){
    var script = document.createElement('script'),
    fn = callback || function(){};
    script.type = 'text/javascript';
    script.onload = function(){
        fn();
    };
    script.src = url;
    document.getElementsByTagName('head')[0].appendChild(script);
}

// 加载opencv
loadJS('https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/opencv/opencv.js', function(){
    console.log('js load ok');
});

// 加载推理引擎onnxruntime.js。当然也可以使用其他推理引擎
loadJS('https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/onnxruntime/onnxruntime.min.js', function(){
    console.log('js load ok');
});

// 初始化python运行环境
loadJS('https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/pyodide/pyodide.js', function(){
    console.log('js load ok');
});

pyodide = await loadPyodide({ indexURL : "https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/pyodide/0.18.0/" });
await pyodide.loadPackage(['micropip']);

Python initialization complete
Loading distutils
Loading distutils from https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/pyodide/0.18.0/distutils.js
Loaded distutils
Loading micropip, pyparsing, packaging, distutils
Loading micropip from https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/pyodide/0.18.0/micropip.js
Loading pyparsing from https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/pyodide/0.18.0/pyparsing.js
Loading packaging from https://test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com/public/pyodide/0.18.0/packaging.js
distutils already loaded from default channel
Loaded micropip, pyparsing, packaging, distutils
```

至此，python和pip就安装完毕了，都位于内存文件系统中。我们可以查看一下python被安装到了哪里：

```
undefined
await pyodide.runPythonAsync(`
print(os.listdir('/home'))
`);
['web_user']
undefined
await pyodide.runPythonAsync(`
print(os.listdir('/home/web_user'))
`);
[]
undefined
await pyodide.runPythonAsync(`
print(os.listdir('/lib'))
`);
['python3.9']
undefined
await pyodide.runPythonAsync(`
print(os.listdir('/lib/python3.9'))
`);
['site-packages', 'importlib', 'asyncio', 'collections', 'concurrent', 'encodings', 'email', 'html', 'json', 'http', 'xmlrpc', 'sqlite3', 'log
'tzdata', 'pydoc_data', 'zoneinfo', 'tzdata-2021.1.dist-info', '_future_.py', '_phello_.foo.py', '_aix_support.py', '_bootlocale.py', '_bc
_py_abc.py', '_pydecimal.py', '_pyio.py', '_sitebuiltins.py', '_strptime.py', '_threading_local.py', '_weakrefset.py', 'abc.py', 'aifc.py', '
'binhex.py', 'bisect.py', 'bz2.py', 'cProfile.py', 'calendar.py', 'cgi.py', 'cgitb.py', 'chunk.py', 'cmd.py', 'code.py', 'codecs.py', 'codeop
'copy.py', 'copyreg.py', 'crypt.py', 'csv.py', 'dataclasses.py', 'datetime.py', 'decimal.py', 'difflib.py', 'dis.py', 'doctest.py', 'enum.py',
'functools.py', 'genericpath.py', 'getopt.py', 'gettext.py', 'glob.py', 'graphlib.py', 'grp.py', 'hashlib.py', 'heapq.py', 'hmac
'linecache.py', 'locale.py', 'lzma.py', 'mailbox.py', 'mailcap.py', 'mimetypes.py', 'modulefinder.py', 'netrc.py', 'ntpath.py', 'ntpath.py',
'pdb.py', 'pickle.py', 'pickletools.py', 'pipes.py', 'pkgutil.py', 'platform.py', 'plistlib.py', 'poplib.py', 'posixpath.py', 'pprint.py', 'pr
'random.py', 're.py', 'reprlib.py', 'rlcompleter.py', 'runpy.py', 'sched.py', 'secrets.py', 'selectors.py', 'shelve.py', 'shlex.py', 'shutil.
'sre_compile.py', 'sre_constants.py', 'sre_parse.py', 'ssl.py', 'stat.py', 'statistics.py', 'string.py', 'stringprep.py', 'struct.py', 'subprc
'telnetlib.py', 'tempfile.py', 'textwrap.py', 'this.py', 'threading.py', 'timeit.py', 'token.py', 'tokenize.py', 'trace.py', 'traceback.py', '
'weakref.py', 'xdrlib.py', 'zipapp.py', 'zipfile.py', 'zipimport.py', 'LICENSE.txt', '_sysconfigdata_emscripten.py', 'webbrowser.py', '_test
```

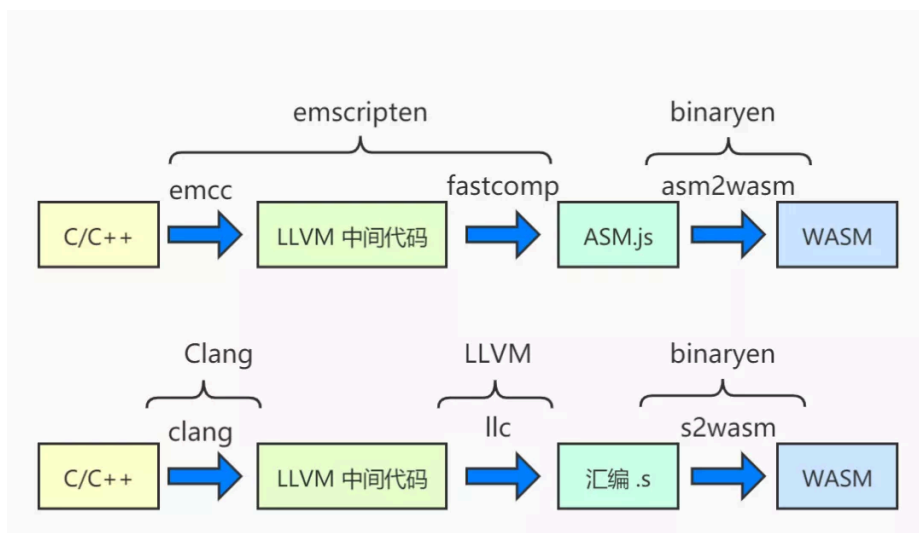
注意，这个文件系统是内存里虚拟出来的，刷新页面就丢失了。不过由于浏览器本身有缓存，所以刷新页面后从服务端再次加载pyodide的引导js和主体wasm还是比较快的，只要不清理浏览器缓存。

2 加载pypi包

在pyodide初始化完成后，python系统自带的标准模块可以直接import。第三方模块需要用micropip.install()安装：

- pypi.org上的纯python包可以用micropip.install() 直接安装
- 含有C语言扩展（编译为[动态链接库](#)⁺）的wheel包，需要对照官方已编译包的列表
 - 在列表中的直接用micropip.install()安装
 - 不在这个列表里的，就需要自己手动编译后发布到服务器后再用micropip.install()安装。

下图展示了业内常用的两种编译为wasm的方式。



自己编译wasm package的方法可参考官方手册，大致步骤就是pull官方的编译基础镜像，把待编译包的setup.cfg⁺文件放到模块目录里，再加上些hack的语句和配置（如果有的话），然后指定目标进行编译。编译成功后部署时，需要注意2点：

- 设置允许跨域
- 对于wasm格式的文件请求，响应Header里应当带上："Content-type": "application/wasm"

下面是一个自建wasm服务器的nginx/openresty示例配置：


```

        add_header 'Access-Control-Allow-Credentials' "true";
        root /path/to/wasm_dir;
        header_filter_by_lua '
            uri = ngx.var.uri
            if string.match(uri, ".js$") == nil then
                ngx.header["Content-type"] = "application/wasm"
            end
        ';
    }
}

```

回到我们的推理实例，现在用pip安装[模型推理](#)所需的numpy和Pillow包并将其import：

```

await pyodide.runPythonAsync(`
import micropip
micropip.install(["numpy", "Pillow"])
`);

await pyodide.runPythonAsync(`
import pyodide
import js.cv as cv2
import js.onnx as onnxruntime
import numpy as np
`);

```

这样python所需的opencv、onnxruntime包就已全部导入了。

3 opencv的使用

一般python里的图片数组都是从JS里传过来的，这里我们模拟构造一张图片，然后用opencv对其resize。上面提到过，pyodide官方的opencv还没编译出来。如果涉及到的opencv方法调用有其他pypi包的替代品，那是最好的：比如，[cv.resize](#)可以用Pillow库的PIL.resize代替（注意Pillow的resize速度比opencv的resize要慢）；cv.threshold可以用numpy.where代替。否则只能调用opencv.js的能力了。为了演示pyodide语法，这里都从opencv.js库里调用。

```

await pyodide.runPythonAsync(`
# 构造一个1080p图片
h,w = 1080,1920
img = np.arange(h * w * 3, dtype=np.uint8).reshape(h, w, 3)

# 使用cv2.resize将其缩小为1/10
# 原python代码: small_img = cv2.resize(img, (h_small, w_small))
# 改成调用opencv.js:
h_small,w_small = 108, 192
mat = cv2.matFromArray(h, w, cv2.CV_8UC3, pyodide.to_js(img.reshape(h * w * 3)))
dst = cv2.Mat.new(h_small, w_small, cv2.CV_8UC3)
cv2.resize(mat, dst, cv2.Size.new(w_small, h_small), 0, 0, cv2.INTER_NEAREST)
small_img = np.asarray(dst.data.to_py()).reshape(h_small, w_small, 3)
`);

```

传参原则：除了简单的数字、字符串类型可以直接传，其他类型都需要通过pyodide.to_js()转换后再传入。返回值的获取也类似，除了简单的数字、字符串类型可以直接获取，其他类型都需要通过xx.to_py()转换后获取结果。

接着对一个mask检测其轮廓：

```

await pyodide.runPythonAsync(`
# 使用cv2.findContours来检测轮廓。假设mask为二维numpy数组，只有0、1两个值
# 原python代码: contours = cv2.findContours(mask, cv2.RETR_CCOMP, cv2.CHAIN_APPROX_SIMPLE)
# 改成调用opencv.js:
contours,
hierarch

```

```
# contours js格式转python格式
contours = []
for i in range(contours_jsproxy.size()):
    c_jsproxy = contours_jsproxy.get(i)
    c = np.asarray(c_jsproxy.data32S.to_py()).reshape(c_jsproxy.rows, c_jsproxy.cols)
    contours.append(c)
`);
```

4 推理引擎的使用

最后，用onnx.js加载模型并进行推理，详细语法可参考onnx.js官方文档。其他js版的推理引擎也都可以参考各自的文档。

```
await pyodide.runPythonAsync(`
model_url="onnx模型的地址"
session = onnxruntime.InferenceSession.new()
session.loadModel(model_url)
session.run(.....)
`);
```

通过以上的操作，我们确保了一切都在python语法范围内进行，这样修改原始的Python文件就比较容易了：把不支持的函数替换成我们自定义的调用js的方法；原Python里的推理替换成调用js版的[推理引擎](#)；最后在Javascript主程序框架里加少许调用Python的胶水代码就完成了。

5 挂载持久存储文件系统

有时我们需要对一些数据持久保存，可以利用pyodide提供的持久化文件系统（其实是emscripten提供的），见手册（文章底部）。

```
// 创建挂载点
pyodide.FS.mkdir('/mnt');
// 挂载文件系统
pyodide.FS.mount(pyodide.FS.filesystems.IDBFS, {}, '/mnt');
// 写入一个文件
pyodide.FS.writeFile('/mnt/test.txt', 'hello world');
// 真正的保存文件到持久文件系统
pyodide.FS.syncfs(function (err) {
    console.log(err);
});
```

这样文件就持久保存了。即使当我们刷新页面后，仍可以通过挂载该文件系统来读出里面的内容：

```
// 创建挂载点
pyodide.FS.mkdir('/mnt');
// 挂载文件系统
pyodide.FS.mount(pyodide.FS.filesystems.IDBFS, {}, '/mnt');
// 写入一个文件
pyodide.FS.writeFile('/mnt/test.txt', 'hello world');
// 真正的保存文件到持久文件系统
pyodide.FS.syncfs(function (err) {
    console.log(err);
});
```

运行结果如下：


```
pyodide.FS.syncfs(true, function (err) {  
  console.log(err);  
});  
undefined  
null  
pyodide.FS.readdir('/mnt');  
▶ (3) [".", "..", "test.txt"]  
pyodide.FS.readFile('/mnt/test.txt', {encoding: 'utf8'});  
"hello world"
```

当然，以上语句可以在python中以Proxy的语法方式运行。

持久文件系统有很多用处。例如，可以帮我们在多线程（webworker）之间共享大数据；可以把模型文件持久存储到文件系统里，无需每次都通过网络加载。

6 打wheel包

单Python文件无需打包，直接当成一个巨大的字符串，交给pyodide.runPythonAsync()运行就好了。当有多个Python文件时，我们可以把这些python文件打成普通wheel包，部署到webserver，然后可以用micropip直接安装该wheel包：

```
micropip.install("https://foo.com/bar-1.2.3-xxx.whl")  
from bar import ...
```

注意，打wheel包需要有__init__.py文件，哪怕是个空文件。

四 存在的缺陷

目前pyodide有如下几个缺陷：

- Python运行环境加载和初始化时间有点儿长，视网络情况，几秒到几十秒都有可能。
- pypi包支持的不完整。虽然pypi.org上的纯python包都可以直接使用，但涉及到C扩展写的包，如果官方还没编译出来，那就需要自己动手编译了。
- 个别很常用的包，例如opencv，还没成功编译出来；模型推理框架一个都没有。不过还好可以通过相应的JS库来弥补。
- 如果python中调用了js库的话：
 - 可能会产生一定的内存拷贝开销（从wasm内存到JS内存的来回拷贝）。尤其是大数组作为参数或返回值，在速度要求高的场合下，额外的内存拷贝开销就不能忽视了。
 - python库的方法接口可能跟其对应的js库的接口参数、返回值格式不一致，有一定的适配工作量。

五 总结

尽管有上述种种缺陷，得益于代码移植的高效率和逻辑上1:1复刻的高可靠性保障，我们还是可以把这种方法运用到多种业务场景里，为推动机器学习技术的应用添砖加瓦。

链接：

1、测试页面：

test-bucket-duplicate.oss-cn-hangzhou.aliyuncs.com...

2、文档：

pyodide.org/en/stable/u...

3、官方已编译包⁺的列表：

github.com/pyodide/pyod...

4、手册：

持久化储存训练营

“Kubernetes 难点攻破训练营系列”的初心是和开发者们一起应对学习和使用 K8s的挑战。这一次，我们从容器持久化存储开始。

[点击这里](#)，查看详情~

版权声明：本文内容由阿里云实名注册用户自发贡献，版权归原作者所有，[阿里云开发者社区](#)不拥有其著作权，亦不承担相应法律责任。具体规则请查看《[阿里云开发者社区用户服务协议](#)》和《[阿里云开发者社区知识产权保护指引](#)》。如果您发现本社区中有涉嫌抄袭的内容，填写[侵权投诉表单](#)进行举报，一经查实，本社区将立刻删除涉嫌侵权内容。

所属专栏 · 2024-12-17 10:42 更新



内容精选合集



阿里云开发者



已认证账号

1949 篇内容 · 27195 赞同

订阅

最热内容 · [如何规范你的Git commit?](#)

发布于 2021-09-30 13:42



理性发言，友善互动



还没有评论，发表第一个评论吧

推荐阅读

复杂推理模型从服务器移植到Web浏览器的理论和实战

一 背景随着机器学习的应用面越来越广，能在浏览器中跑模型推理的Javascript框架引擎也越来越多的。在项目中，前端同学可能会找到一些跑在服务端的python算法模型，很想将其直接集成到自己...

阿里云栖... 发表于二栖技术网

DeepSeek-R1 \ Kimi 1.5 及类强推理模型开发解读

陈博远
北京大学2022级“通信”
主要研究方向：大语言模型对齐与可控生成
<https://bygh.github.io/>
<https://pku-gh.com/>

北京大学 | 第三讲
《DeepSeek-R1及类强推理...

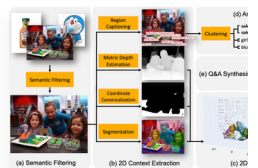
瑞其尚 AI



triton-inference-server的
backend (一) ——关于推理...

AI DBA

发表于深度学习网



SpatialVLM: 赋予视觉
型空间推理能力

蔡汝

发表于:

